

AI DAY 1: PYTHON + DATA FOUNDATIONS

PART 1: Why Python for AI?

👉 Because:

- Easy syntax 
 - Huge ML libraries 
 - Industry standard 
-

PART 2: Python Basics (Fast Track)

Variables

```
Python
name = "Sameer"
age = 25
Show more lines
```

List

```
Python
skills = ["Java", "Spring", "AI"]
print(skills[0])
Show more lines
```

✓ Dictionary

Python

```
user = {  
    "name": "Sameer",  
    "role": "Developer"  
}
```

```
print(user["name"])
```

Show more lines

✓ Loop

Python

```
for i in range(5):  
    print(i)
```

Show more lines

✓ Function

Python

```
def greet(name):  
    return "Hello " + name
```

```
print(greet("Sameer"))
```

Show more lines

✓ PART 3: NumPy (Basic)

👉 Used for fast calculations

Python

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4])  
print(arr.mean())
```

Show more lines

✅ **PART 4: Pandas** 🔥 🔥 (MOST IMPORTANT)

👉 Used for real data handling

✅ **Load Data**

Python

```
import pandas as pd
```

```
data = pd.read_csv("data.csv")  
print(data.head())
```

Show more lines

✅ **Select Column**

Python

```
print(data["name"])
```

Show more lines

✅ **Filter Data**

Python

```
high_salary = data[data["salary"] > 50000]  
print(high_salary)
```

Show more lines

Add Column

Python

```
data["bonus"] = data["salary"] * 0.1
```

Show more lines

PART 5: Visualization

Python

```
import matplotlib.pyplot as plt
```

```
plt.plot([1, 2, 3], [10, 20, 30])
```

```
plt.show()
```

Show more lines

PART 6: Real Example

Python

```
import pandas as pd
```

```
data = {  
    "name": ["A", "B", "C"],  
    "salary": [40000, 60000, 80000]  
}
```

```
df = pd.DataFrame(data)
```

```
# filter
```

```
high = df[df["salary"] > 50000]
```

```
print(high)
```

Show more lines



INTERVIEW QUESTIONS

1. Why Python is used in AI?
 2. What is Pandas?
 3. Difference: list vs numpy array?
-



MINI TASK (DO THIS)



Task 1:

Create list:

None

```
["AI", "ML", "Cloud"]
```

Print all



Task 2:

Create dictionary:

None

```
name, skill
```



Task 3:

Use pandas:

- Create DataFrame
- Filter salary > 50000

AI DAY 2: DATA CLEANING + ADVANCED PANDAS

Goal

By end of today you will:  Work with real datasets

 Clean messy data (VERY IMPORTANT )

 Prepare data for ML models

 Think like a data engineer

PART 1: Why Data Cleaning is IMPORTANT?

 In real world:

 Data is NEVER clean

 Missing values

 Wrong values

 Duplicate data

Real Truth:

 **80% work in AI = Data cleaning**

PART 2: Load Real Dataset

Python

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
```

```
print(df.head())
```

Show more lines

Check Data Info

Python

```
print(df.info())
```

Show more lines

👉 Shows:

- Columns
 - Null values
 - Data types
-

PART 3: Detect Missing Values

Python

```
print(df.isnull().sum())
```

Show more lines

 **Example Output:**

None

name 0

salary 2

age 1

👉 Means missing data ❌

✅ PART 4: Handle Missing Values

✅ Option 1: Remove Missing Rows

Python

```
df = df.dropna()
```

Show more lines

👉 Deletes rows with missing data

✅ Option 2: Fill Missing Values ✅ (Better)

Python

```
df["salary"].fillna(df["salary"].mean(), inplace=True)
```

Show more lines

👉 Replace missing with average ✅

✅ PART 5: Remove Duplicate Data

Python

```
df = df.drop_duplicates()
```

Show more lines

👉 Important in real datasets ✓

✓ PART 6: Filter Data 🔥

✓ Example:

Python

```
high_salary = df[df["salary"] > 50000]
```

```
print(high_salary)
```

Show more lines

👉 This is heavily used in ML ✓

✓ PART 7: Sorting Data

Python

```
df = df.sort_values(by="salary", ascending=False)
```

Show more lines

✓ PART 8: Grouping Data 🔥 🔥

👉 VERY IMPORTANT (used in analytics)

✓ Example:

Python

```
grouped = df.groupby("department")["salary"].mean()  
print(grouped)
```

Show more lines

👉 Output:

None	
IT	60000
HR	45000

✓ PART 9: Feature Engineering 🔥 🔥 🔥

👉 Create new features from existing data

✓ Example:

Python

```
df["bonus"] = df["salary"] * 0.1
```

Show more lines

👉 This improves ML performance ✓

✓ PART 10: Convert Data Types

Python

```
df["age"] = df["age"].astype(int)
```

Show more lines

👉 Important for ML models ✓

✓ PART 11: Basic Visualization 🔥

Python

```
import matplotlib.pyplot as plt
```

```
df["salary"].hist()
```

```
plt.show()
```

Show more lines

👉 Helps understand data distribution ✓

✓ PART 12: End-to-End Data Cleaning

Example 🔥

Python

```
import pandas as pd
```

```
# create sample data
```

```
data = {
```

```
"name": ["A", "B", "C", None],
```

```
"salary": [40000, None, 80000, 60000],
```

```
"department": ["IT", "HR", "IT", "HR"]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# check nulls  
print(df.isnull().sum())
```

```
# fill missing salary  
df["salary"].fillna(df["salary"].mean(), inplace=True)
```

```
# drop missing name  
df = df.dropna()
```

```
# add bonus column  
df["bonus"] = df["salary"] * 0.1
```

```
print(df)
```

Show more lines

PART 13: What You Learned Today

-  Handling missing values
 -  Cleaning dataset
 -  Feature creation
 -  Data filtering
 -  Grouping
-

 This is **REAL data science work**

INTERVIEW QUESTIONS

1. What is data cleaning?
 2. How do you handle missing values?
 3. What is feature engineering?
 4. What is `groupby()`?
 5. Why data cleaning is important?
-

MINI TASK (IMPORTANT — DO THIS)

Task 1:

Create DataFrame:

```
None  
name, salary  
(include some NULL values)
```

✓ Fill missing salary with mean

Task 2:

Filter: 👉 salary > 50000

Task 3:

Group by department
→ find average salary

🔥 Task 4 (PRO 🔥 🔥):

Add new column:

None

```
tax = salary * 0.2
```

🚀 🧠 AI DAY 3: MACHINE LEARNING BASICS + FIRST MODEL

🎯 Goal

By end of today, you will: ☒ Understand what Machine Learning is

☒ Learn training vs testing

☒ Build your first ML model ☒

☒ Make predictions

☒ PART 1: What is Machine Learning?

👉 ML = Teaching computers to learn from data

🧠 Example

Instead of writing:

Java

if (experience > 5) salary = high;

Show more lines

👉 ML will **learn pattern automatically** from data ✓

✓ Real-world Examples

- Netflix recommendations
 - Spam detection
 - Salary prediction
-

✓ PART 2: Types of ML (IMPORTANT)

✓ 1. Supervised Learning ✓ (YOU START HERE)

👉 Data has input + output

Example:

None

Experience → Salary

✓ 2. Unsupervised Learning

👉 No output

Example:

- Group customers
-

✓ 3. Reinforcement Learning

👉 Learn via rewards (games, AI bots)

👉 Focus: ☒ Supervised learning

☒ PART 3: Key Terms 🔥

☒ 1. Features (Input)

None

experience, age

☒ 2. Target (Output)

None

salary

☒ 3. Model

👉 Algorithm that learns relationship

☒ 4. Training

👉 Teaching model using data

✓ 5. Prediction

👉 Model gives output

✓ PART 4: Training vs Testing 🔥 🔥 🔥

🧠 Why needed?

👉 To check model performance

✓ Example:

None

```
100 rows data  
→ 80% training  
→ 20% testing
```

✓ Train on 80

✓ Test on 20

👉 Ensures model is correct ✓

✓ PART 5: First Algorithm — Linear Regression 🔥

👉 Used for predicting numbers

✅ Example

None

Experience → Salary

✅ Simple Formula Understanding

👉 ML tries to fit a line:

None

$\text{salary} = m \times \text{experience} + c$

👉 Don't worry about math ✅

👉 Focus on usage ✅

✅ PART 6: YOUR FIRST MODEL CODE



Python

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
# Step 1: Create Data
```

```
data = {
```

```
"experience": [1, 2, 3, 4, 5],
```

```
"salary": [30000, 40000, 50000, 60000, 70000]
}

df = pd.DataFrame(data)

# Step 2: Features & Target
X = df[["experience"]] # input
y = df["salary"] # output

# Step 3: Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Step 4: Create model
model = LinearRegression()

# Step 5: Train model
model.fit(X_train, y_train)

# Step 6: Predict
prediction = model.predict([[6]])

print("Predicted Salary:", prediction)
Show more lines
```

PART 7: Output Example

None

Predicted Salary: ~80000

👍  Model learned pattern

👍  Predicts new value

✓ PART 8: Model Accuracy (Basic)

Python

```
score = model.score(X_test, y_test)
```

```
print("Accuracy:", score)
```

Show more lines

👉 Value close to 1 = good model ✓

✓ PART 9: Overfitting vs Underfitting 🔥

✗ Overfitting

👉 Model memorizes data

👉 Fails on new data

✗ Underfitting

👉 Model too simple

👉 Poor predictions

✓ Ideal: Balance

✓ PART 10: Real Understanding Flow 🔥

None

1. Load Data
2. Clean Data ☒ (Day 2)
3. Split Data
4. Train Model
5. Test Model
6. Predict

👉 This is **real ML pipeline**

☒ **PART 11: Interview Questions** 🔥

1. What is machine learning?
 2. What is supervised learning?
 3. What is training vs testing?
 4. What is linear regression?
 5. What is overfitting?
-

☒ **MINI TASK (VERY IMPORTANT)**

🔥 **Task 1:**

Create dataset:

None

hours_studied → marks

Train model ☒

Task 2:

Predict marks for:

None

6 hours

Task 3:

Print accuracy

Task 4 (PRO):

Modify:

- Add more data points
- See how prediction improves

AI DAY 4: LOGISTIC REGRESSION + CLASSIFICATION

Goal

By end of today, you will: ☒ Understand classification problems

☒ Learn Logistic Regression (VERY IMPORTANT )

☒ Build your first classification model

☒ Create a **Spam Detector** ☒

✓ PART 1: Regression vs Classification 🔥

Type	Output
Regression	Numbers (salary, price)
Classification	Categories (YES/NO, spam/not spam)

🧠 Example

✓ Regression:

None
`experience → salary`

✓ Classification:

None
`email → spam or not`

👉 Today: ✓ You learn **Classification**

✓ PART 2: What is Logistic Regression?

👉 Used for: ✓ YES / NO decisions

✓ 0 / 1 output

🧠 Example

None

Student pass? → Yes / No

Email spam? → Yes / No

✓ Output:

- 0 = No ✗
 - 1 = Yes ✓
-

✓ PART 3: How it Works (Simple)

👉 Instead of straight line → it uses **S-curve**

None

Probability:

0 → 1

👉 If probability > 0.5 → YES ✓

👉 Else → NO ✗

✓ PART 4: First Classification Example 🔥

✓ Problem:

Predict if student passes

Code

Python

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
# Step 1: Dataset
```

```
data = {
```

```
"hours": [1, 2, 3, 4, 5, 6],
```

```
"pass": [0, 0, 0, 1, 1, 1]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
# Step 2: Features & target
```

```
X = df[["hours"]]
```

```
y = df["pass"]
```

```
# Step 3: Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Step 4: Model
```

```
model = LogisticRegression()
```

```
# Step 5: Train
```

```
model.fit(X_train, y_train)
```

```
# Step 6: Predict
```

```
prediction = model.predict([[3]])
```

```
print("Pass or Not:", prediction)
```

```
Show more lines
```

 Output:

None

[0] → Not pass

✓ PART 5: Real Example — SPAM DETECTION 🔥 🔥

✓ Problem

Classify email:

None

Spam → 1

Not Spam → 0

✓ Example Dataset Concept

message	spam
win money now	1
meeting at 5	0

✓ Basic Code Concept

Python

```
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.linear_model import LogisticRegression
```

```
# sample data
```

```
messages = ["win money now", "call me", "free lottery", "meeting at office"]  
labels = [1, 0, 1, 0]
```

```
# convert text to numbers  
vectorizer = CountVectorizer()  
X = vectorizer.fit_transform(messages)
```

```
# model  
model = LogisticRegression()  
model.fit(X, labels)
```

```
# predict  
test = vectorizer.transform(["free money"])  
print(model.predict(test))  
Show more lines
```

👉 Output:

None

[1] → Spam ✓

✓ PART 6: Accuracy Metrics 🔥 🔥

✓ Accuracy

Python
model.score(X_test, y_test)
Show more lines

✓ Confusion Matrix (IMPORTANT)

Actual	Predicted
TP	Correct YES
TN	Correct NO
FP	Wrong YES
FN	Wrong NO

✓ Precision & Recall (INTERVIEW)

Metric	Meaning
Precision	Correct among predicted YES
Recall	Correct among actual YES

✓ PART 7: When to Use Logistic Regression?

- ✓ Binary classification
 - ✓ Simple problems
 - ✓ Fast models
-

✓ PART 8: Real-world Use Cases 🔥

- Spam detection ✓
- Fraud detection ✓

- Medical diagnosis ✓
 - Customer churn prediction ✓
-

✓ PART 9: ML Pipeline Update 🔥

Now your pipeline becomes:

None

1. Load data
 2. Clean data ✓
 3. Feature conversion
 4. Train model
 5. Evaluate
 6. Predict
-

✓ PART 10: Interview Questions 🔥

1. Difference: Linear vs Logistic Regression?
 2. What is classification?
 3. What is sigmoid function?
 4. What is precision vs recall?
 5. What is confusion matrix?
-

✓ MINI TASK (VERY IMPORTANT)

🔥 Task 1:

Create dataset:

None

hours → pass/fail

Train logistic model ✓

🔥 Task 2:

Predict:

None

5 hours → pass?

🔥 Task 3:

Build simple spam classifier ✓

🔥 Task 4 (PRO 🔥 🔥):

Try:

- Add more messages
- Improve accuracy

🚀 🧠 AI DAY 5: DECISION TREE + RANDOM FOREST

🎯 Goal

By end of today, you will: ☒ Understand Decision Trees

☒ Learn Random Forest (VERY IMPORTANT 🔥)

☒ Build better ML models

☒ Improve accuracy

☒ PART 1: Problem with Previous Models

👉 Linear & Logistic Regression:

❌ Don't handle complex patterns

❌ Assume linear relationship

👉 Need something smarter → ☒ Decision Trees

☒ PART 2: What is Decision Tree? 🌳

👉 A model that works like **if-else conditions**

🧠 Real-Life Analogy

None

Is salary > 50K?

YES → High income

NO → Low income

👉 Like a tree structure:

None

```
(salary > 50K?)  
  /      \  
Yes       No  
High     Low
```

- ✓ Easy to understand
- ✓ Works with complex data

✓ PART 3: Example Problem

✓ Dataset

age	salary	buy
25	30000	0
35	60000	1

👉 Model creates rules:

None

```
IF salary > 50000 → buy = YES
```

✓ PART 4: Decision Tree Code 🔥

Python

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
```

```
# dataset
```

```
data = {
    "age": [25, 30, 35, 40],
    "salary": [30000, 40000, 60000, 70000],
    "buy": [0, 0, 1, 1]
}
```

```
df = pd.DataFrame(data)
```

```
X = df[["age", "salary"]]
y = df["buy"]
```

```
# model
```

```
model = DecisionTreeClassifier()
```

```
# train
```

```
model.fit(X, y)
```

```
# predict
```

```
prediction = model.predict([[32, 50000]])
```

```
print("Will Buy:", prediction)
```

```
Show more lines
```

👉 Output:

None

[0] → No

PART 5: Advantages of Decision Trees

- ✓ Easy to understand
 - ✓ Handles complex patterns
 - ✓ No need data scaling
-

⚠ **Problem:**

- ✗ Overfitting (learns too much)
-

✓ **PART 6: Random Forest** 🔥 🔥 🔥 **(VERY IMPORTANT)**

- 👉 Solution to overfitting ✓
-

🧠 **What is Random Forest?**

- 👉 Multiple Decision Trees combined
-

🧠 **Analogy**

- 👉 Instead of 1 person's opinion
 - 👉 Ask 100 people → take majority decision ✓
-

- ✓ More accurate
 - ✓ More stable
-

✓ PART 7: Random Forest Code 🔥

Python

```
from sklearn.ensemble import RandomForestClassifier
```

```
# model
```

```
model = RandomForestClassifier(n_estimators=10)
```

```
# train
```

```
model.fit(X, y)
```

```
# predict
```

```
prediction = model.predict([[32, 50000]])
```

```
print("Will Buy:", prediction)
```

Show more lines

✓ PART 8: Why Random Forest is Powerful?

✓ Reduces overfitting

✓ Works well in real-world data

✓ High accuracy

✓ PART 9: When to Use What?

Model	Use Case
-------	----------

Linear Regression	Simple prediction
Logistic Regression	Binary classification
Decision Tree	Rule-based decision
Random Forest	Real-world complex data 

PART 10: Feature Importance

 Shows which feature matters most

Python

```
print(model.feature_importances_)
```

Show more lines

 Example output:

None

 [0.3, 0.7]

 Salary is more important than age 

PART 11: Real-world Use Cases

-  Credit approval
-  Fraud detection
-  Recommendation systems
-  Customer segmentation

✓ PART 12: Full ML Pipeline (Updated)

None

1. Data Collection
2. Data Cleaning ✓
3. Feature Engineering ✓
4. Model Selection ✓
5. Training ✓
6. Evaluation ✓
7. Prediction ✓

👉 You are now understanding FULL pipeline ✓

✓ PART 13: Interview Questions 🔥

1. What is Decision Tree?
 2. What is Random Forest?
 3. Why Random Forest is better?
 4. What is overfitting?
 5. What is feature importance?
-

✓ MINI TASK (VERY IMPORTANT)

🔥 Task 1:

Use Decision Tree:

None

age, salary → buy

Task 2:

Use Random Forest ☒

Task 3:

Compare accuracy:

- Tree vs Forest
-

Task 4 (PRO):

Print feature importance ☒

AI DAY 6: KNN + MODEL TUNING

Goal

By end of today, you will: ☒ Understand KNN algorithm

☒ Learn how ML makes decisions using neighbors

☒ Improve model performance

☒ Understand tuning concepts

✓ PART 1: What is KNN? 🔥

👉 KNN = K Nearest Neighbors

👉 It predicts based on **similar data points**

🧠 Real-Life Analogy

👉 Suppose:

- Your 5 friends love cricket 🏏
👉 Then you will likely also like cricket

👉 Decision based on **neighbors** ✓

✓ PART 2: How KNN Works

✓ Steps:

None

1. Choose K (number of neighbors)
 2. Find closest data points
 3. Majority decision
-

🧠 Example:

None

K = 3

Neighbors:
YES, YES, NO

→ Final = YES ✓

✓ PART 3: KNN Code 🔥

Python

```
import pandas as pd
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
# dataset
```

```
data = {
```

```
"age": [20, 25, 30, 35],
```

```
"salary": [20000, 30000, 40000, 50000],
```

```
"buy": [0, 0, 1, 1]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
X = df[["age", "salary"]]
```

```
y = df["buy"]
```

```
# model
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
# train
```

```
model.fit(X, y)
```

```
# predict
```

```
prediction = model.predict([[28, 35000]])
```

```
print("Will Buy:", prediction)
```

Show more lines

✓ Output:

None

[1] → YES

✓ PART 4: Choosing K Value 🔥

🧠 Important Rule:

K value	Result
Small K	Noisy, overfitting
Large K	Smooth, underfitting

👉 Best practice: ✓ Try multiple values (3, 5, 7)

✓ PART 5: Problem with KNN ⚠️

✗ Slow for large data

✗ Sensitive to scale

✓ Solution: Feature Scaling 🔥

✓ PART 6: Feature Scaling

👉 Normalize data

Python

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Show more lines

✓ Improves accuracy ✓

✓ PART 7: Model Evaluation 🔥

✓ Accuracy

Python

```
model.score(X, y)
```

Show more lines

✓ Compare Models

👉 Try:

- Logistic Regression
 - Decision Tree
 - Random Forest
 - KNN
-

✓ Choose best accuracy ✓

✓ **PART 8: Model Tuning** 🔥 🔥 🔥

👉 Improve performance by adjusting parameters

✓ **Example: Random Forest**

Python

```
model = RandomForestClassifier(n_estimators=100, max_depth=5)
```

Show more lines

✓ **Example: KNN**

Python

```
model = KNeighborsClassifier(n_neighbors=5)
```

Show more lines

👉 Changing parameters = **Tuning** ✓

✓ **PART 9: Cross Validation (ADVANCED)**

👉 Test model multiple times

Python

```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(model, X, y, cv=5)
```

```
print(scores.mean())
```

Show more lines

✅ More reliable accuracy ✅

✅ PART 10: When to Use KNN?

✅ Small datasets

✅ Pattern-based problems

❌ Avoid for:

- Large data
 - Real-time systems
-

✅ PART 11: Model Comparison 🔥

Model	Strength
Logistic	Fast
Decision Tree	Simple
Random Forest	Accurate ✅
KNN	Pattern-based

👉 Best general-purpose: ✅ Random Forest 🔥

✓ PART 12: Real Pipeline (FINAL)

None

1. Load data
2. Clean data ✓
3. Scale data ✓
4. Train model ✓
5. Tune model ✓
6. Evaluate ✓
7. Deploy (next phase)

👉 You now understand full ML workflow ✓

✓ PART 13: Interview Questions 🔥

1. What is KNN?
 2. How does KNN work?
 3. What is K value?
 4. Why scaling needed?
 5. What is model tuning?
 6. What is cross-validation?
-

✓ MINI TASK (VERY IMPORTANT)

🔥 Task 1:

Use KNN:

None

age, salary → buy

Task 2:

Try different K values:

None

K = 3, 5, 7

Task 3:

Apply scaling ☒

Task 4 (PRO):

Compare:

- Logistic
- Decision Tree
- Random Forest
- KNN

AI DAY 7: MODEL DEPLOYMENT + REAL USAGE

Goal

By end of today, you will: ☒ Save ML model ☒

☒ Load model ☒

- ✓ Use model in real app ✓
 - ✓ Connect ML → Backend (Spring Boot concept) ✓
-

✓ PART 1: Why Deployment is IMPORTANT?

👉 ML without deployment = ✗ useless

Companies want: ✓ “Can you use ML in real apps?”

🧠 Example

- Predict salary ✓
- But user cannot access ✗

👉 So we: ✓ deploy model

✓ expose via API

✓ PART 2: Save Trained Model 🔥

👉 Use `joblib` / `pickle`

✓ Save Model

Python
`import joblib`

```
joblib.dump(model, "model.pkl")
```

Show more lines

✅ This saves trained model → file ✅

✅ PART 3: Load Model

Python

```
model = joblib.load("model.pkl")
```

Show more lines

✅ Now you can reuse without training ✅

✅ PART 4: Predict Using Saved Model

Python

```
prediction = model.predict([[5]])
```

```
print(prediction)
```

Show more lines

👉 ✅ Same result without retraining

👉 ✅ Production-ready

✅ PART 5: Build Simple API (Python) 🔥

👉 Use **Flask** (lightweight backend)

Install Flask

Shell

```
pip install flask
```

Show more lines

API Code

Python

```
from flask import Flask, request, jsonify
import joblib
```

```
app = Flask(__name__)
```

```
model = joblib.load("model.pkl")
```

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    data = request.json
```

```
    value = data["experience"]
```

```
    prediction = model.predict([[value]])
```

```
    return jsonify({"prediction": prediction.tolist()})
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```

Show more lines

Call API

POST `/predict`

JSON

```
{
  "experience": 5
}
```

```
}
```

Show more lines

👉 Response:

JSON

```
{
```

```
"prediction": [70000]
```

```
}
```

Show more lines

✅ ML model is now API ✅

✅ PART 6: Connect with Spring Boot



👉 THIS is your superpower 💥

✅ Flow

None

Frontend (React)

↓

Spring Boot API

↓

Call Python ML API

↓

Return prediction

✓ Java Call Example (Concept)

Java

```
RestTemplate restTemplate = new RestTemplate();
```

```
String url = "http://localhost:5000/predict";
```

```
Map<String, Integer> req = new HashMap<>();  
req.put("experience", 5);
```

```
ResponseEntity<String> response = restTemplate.postForEntity(url, req, String.class);
```

```
System.out.println(response.getBody());
```

Show more lines

✓ Now: 👉 Spring Boot + AI working together ✓

✓ PART 7: Alternative (Better Architecture)



👉 Use **microservice approach**

✓ Architecture

None

Frontend (React)

↓

Spring Boot (Backend)

↓

ML Service (Python)

↓

Database (optional)

- ✓ Clean separation
 - ✓ Scalable
-

✓ PART 8: Deploy ML Model 🚀

✓ Option 1 (Easy)

- Render
 - Railway
-

✓ Option 2 (AWS)

- EC2 → run Flask app
 - S3 → store model
-

✓ Now your model is LIVE 🌐

✓ PART 9: REAL PROJECT IDEAS 🔥 🔥 🔥

🔥 Project 1: Salary Predictor App

- Input: experience
 - Output: salary
 - Stack:
 - React
 - Spring Boot
 - Flask ML
-

Project 2: Spam Detector

- Input: message
 - Output: spam/not spam
-

Project 3: AI Resume Analyzer

 BEST for jobs

- Upload resume
 - AI gives feedback
-

Project 4: Recommendation System

- Suggest products
-

PART 10: Complete AI Pipeline

None

1. Data Cleaning 
2. Model Training 
3. Model Saving 
4. API Creation 

5. Backend Integration ✓

6. Deployment ✓

👉 You now know FULL lifecycle ✓

✓ PART 11: Interview Questions 🔥 🔥 🔥

1. How do you deploy ML model?
 2. What is model serialization?
 3. How to connect ML with backend?
 4. Flask vs Spring Boot for ML?
 5. What is microservice architecture?
-

✓ MINI TASK (VERY IMPORTANT)

🔥 Task 1:

Train model → save as `.pkl`

🔥 Task 2:

Create Flask API ✓

🔥 Task 3:

Call API using Postman 

Task 4 (PRO):

Call ML API from Java 

WHAT YOU HAVE ACHIEVED

Sameer — HUGE milestone:

-  Python 
 -  Data Science 
 -  Machine Learning 
 -  Multiple algorithms 
 -  Model tuning 
 -  Model deployment 
 -  Backend integration 
-

 You are now:

AI-ENABLED FULLSTACK ENGINEER

NOW LET'S START — CLOUD DAY 1



CLOUD DAY 1: CORE BASICS + SETUP



PART 1: AWS Account Setup

👉 Go to:

None

<https://aws.amazon.com>

Create free-tier account



PART 2: What is EC2?

👉 Virtual server in cloud



Analogy

EC2 = Your laptop (but in cloud)



PART 3: Create EC2 Instance

Steps:

1. Go to EC2 dashboard
2. Click "Launch Instance"
3. Choose:
 - Ubuntu
4. Select:

- t2.micro (free tier)
5. Create key pair (IMPORTANT 🔥)

✓ PART 4: Connect to EC2

Shell

```
ssh -i key.pem ubuntu@public-ip
```

Show more lines

✓ Now you're inside cloud server

✓ PART 5: Install Java

Shell

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk
```

Show more lines

✓ PART 6: Run First App

Upload JAR:

Shell

```
scp -i key.pem app.jar ubuntu@ip:/home
```

Show more lines

Run:

Shell

```
java -jar app.jar
```

Show more lines

✅ App running in cloud ✅

✅ PART 7: Open Port (IMPORTANT 🔥)

👉 Allow port 8080

Go to:

- Security Groups → Edit inbound rules
- Add:

None

8080 → Allow

👉 Access:

None

http://your-ip:8080

✅ Your backend is LIVE 🌐

✅ MINI TASK (DO THIS)

Task 1:

Create EC2 instance ☒

Task 2:

Connect via SSH ☒

Task 3:

Install Java ☒

Task 4:

Deploy your Spring Boot app ☒

CLOUD DAY 2: RDS + SPRING BOOT INTEGRATION

Goal

By end of today, you will: ☒ Create database on AWS ☒

☒ Connect Spring Boot to cloud DB ☒

☒ Understand real production DB setup ☒

✓ PART 1: What is RDS? 🔥

👉 RDS = **Relational Database Service (AWS)**

🧠 Simple Understanding

Instead of installing MySQL manually:

👉 AWS gives you **ready-made database server**

✓ Supported DBs:

- MySQL ✓
 - PostgreSQL ✓
 - Oracle
-

🧠 Analogy

None

Local MySQL → Laptop DB

RDS → Cloud DB (accessible from anywhere)

✓ PART 2: Why Use RDS?

- ✓ No manual installation
 - ✓ Automatic backups
 - ✓ Scalable
 - ✓ High availability
-

👉 This is what companies use ☒

☒ PART 3: Create RDS Instance (Step-by-Step) 🔥

☒ Steps:

1. Go to AWS Console
 2. Search **RDS**
 3. Click “Create Database”
-

⚙️ Choose:

- Engine: ☒ MySQL
 - Template: ☒ Free Tier
 - DB Name: `mydb`
 - Username: `admin`
 - Password: (set something)
-
-

⚠️ IMPORTANT SETTINGS

- 👉 Public Access → ☒ Enable
 - 👉 VPC security → Allow connections
-

Click **Create DB**

☒ Your DB will start in ~5–10 mins

✓ PART 4: Get DB Connection Details

After creation:

👉 Copy:

None

Endpoint: mydb.abc123.us-east-1.rds.amazonaws.com

Port: 3306

✓ This is your DB URL

✓ PART 5: Allow Access (IMPORTANT 🔥)

👉 Go to Security Group

Edit inbound rules:

None

Type: MySQL/Aurora

Port: 3306

Source: Anywhere (for learning)

✓ Now your app can connect ✓

✓ PART 6: Connect Spring Boot to RDS



✓ application.properties

```
.properties
spring.datasource.url=jdbc:mysql://YOUR-ENDPOINT:3306/mydb
spring.datasource.username=admin
spring.datasource.password=yourpassword
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
Show more lines
```

✓ Replace:

- **YOUR-ENDPOINT** with RDS URL

✓ PART 7: Test Connection ✓

Run your Spring Boot app

👉 If no errors: ✓ Connected successfully 🎉

👉 Now:

- Your APIs use **cloud database**
 - Data persists even if app restarts ✓
-

✓ PART 8: Verify Using MySQL Workbench (Optional)

Connect using:

- Host: RDS endpoint
- Username/password

Run:

SQL

```
SELECT * FROM employee;
```

```
--
```

Show more lines

✓ You'll see real data ✓

✓ PART 9: Real Architecture (Now You Have 🔥)

None

Client

↓

EC2 (Spring Boot App)

↓

RDS (Cloud DB)

👉 This is **real backend architecture**

✓ PART 10: BONUS — Connection Issues (COMMON)

✗ Error: Cannot connect

👉 Fix:

- Check security group ✓
 - Enable public access ✓
-

✗ Wrong credentials

👉 Double-check username/password ✓

✗ Timeout

👉 Port 3306 not open ✗

✓ PART 11: Production Best Practices (IMPORTANT)

- ✓ Don't use public access (only private in real prod)
- ✓ Use environment variables (not plain password)
- ✓ Use connection pooling

✓ **PART 12: What You Achieved** 🔥

Sameer — big milestone:

- ✓ Cloud server (EC2) ✓
- ✓ Cloud DB (RDS) ✓
- ✓ Backend connected ✓

👉 You now know: 🔥 **Real-world backend infrastructure**

✓ **MINI TASK (CRITICAL — DO THIS)**

🔥 **Task 1:**

Create RDS DB ✓

🔥 **Task 2:**

Connect Spring Boot to RDS ✓

🔥 **Task 3:**

Insert data using API ✓

Task 4:

Verify in DB 

CLOUD DAY 3: AWS S3 (FILE STORAGE)

Goal

By end of today, you will:  Understand S3 (cloud storage)

 Upload files to cloud 

 Connect Spring Boot with S3 

 Build real-world feature (file upload) 

PART 1: What is S3?

 S3 = Simple Storage Service

Simple Meaning

 It's a cloud storage bucket

You can store:

- Images 
- Videos 
- PDFs 

- Any file 
-

Analogy

None

Google Drive → S3

 But for applications

PART 2: Why S3 is IMPORTANT?

-  Scalable (millions of files)
 -  High availability
 -  Used in production systems
-

Real Use Cases

- Profile pictures 
 - Resume upload 
 - Product images 
 - Document storage 
-

PART 3: Create S3 Bucket

Steps:

1. Go to AWS console

2. Search **S3**
 3. Click **Create Bucket**
-

Settings:

- Bucket name: `sameer-app-bucket`
- Region: (choose region)
- Keep default settings

 Create 

PART 4: Upload File Manually

1. Open bucket
 2. Click **Upload**
 3. Add file
-

 Now file is stored in cloud 

PART 5: Make File Public (IMPORTANT)

By default:  Files are private

To access:

1. Select file
2. Click "Permissions"

3. Enable **public access**

👉 Get URL:

None

<https://bucket-name.s3.amazonaws.com/file.jpg>

✅ Access anywhere 🌐

✅ **PART 6: Spring Boot + S3 Integration**



👉 Now the important part ⚡

✅ **Step 1: Add Dependency**

XML

```
<dependency>
<groupId>com.amazonaws</groupId>
<artifactId>aws-java-sdk-s3</artifactId>
</dependency>
```

Show more lines

✅ **PART 7: Configure AWS Credentials**

✅ **Option 1: ~/.aws/credentials**

None

```
aws_access_key_id=YOUR_KEY  
aws_secret_access_key=YOUR_SECRET
```

✅ Option 2 (Better for prod):

👉 Use IAM roles

✅ PART 8: S3 Service Code 🔥

Java

```
import com.amazonaws.services.s3.AmazonS3;  
import com.amazonaws.services.s3.AmazonS3ClientBuilder;  
import org.springframework.stereotype.Service;  
  
@Service  
public class S3Service {  
  
    private final AmazonS3 s3 = AmazonS3ClientBuilder.defaultClient();  
  
    private final String bucketName = "sameer-app-bucket";  
  
    public String uploadFile(String fileName, File file) {  
        s3.putObject(bucketName, fileName, file);  
  
        return "https://" + bucketName + ".s3.amazonaws.com/" + fileName;  
    }  
}
```

Show more lines

✓ PART 9: Controller API

Java

```
@PostMapping("/upload")
```

```
public String upload(@RequestParam MultipartFile file) throws Exception {
```

```
    File convFile = new File(file.getOriginalFilename());
```

```
    file.transferTo(convFile);
```

```
    return s3Service.uploadFile(file.getOriginalFilename(), convFile);
```

```
}
```

Show more lines

✓ Input:

- File upload

✓ Output:

- File URL ✓
-

✓ PART 10: Data Flow 🔥

None

User → Upload file

↓

Spring Boot API

↓

S3 Bucket

↓

Return file URL

✓ Now file is stored in cloud ✓

✓ PART 11: Real Architecture 🔥 🔥 🔥

```
None
Frontend (React)
  ↓
Spring Boot API
  ↓
S3 (File Storage)
  ↓
RDS (Metadata if needed)
```

✓ This is **production-level pattern**

✓ PART 12: Store File URL in Database

👉 Best practice:

```
None
Store only URL in DB
```

Example:

```
None
employee
id | name | resume_url
```

✓ Lightweight

✓ Efficient

✓ **PART 13: Common Mistakes** ⚠️

✗ Upload files directly to DB

✗ Not securing bucket

✗ Hardcoding AWS keys

✓ **PART 14: What You Achieved** 🔥

Sameer, now you have:

✓ Cloud Compute (EC2) ✓

✓ Cloud Database (RDS) ✓

✓ Cloud Storage (S3) ✓

👉 This is **REAL** backend infrastructure

✓ **MINI TASK (VERY IMPORTANT)**

🔥 **Task 1:**

Create S3 bucket ✓

Task 2:

Upload file manually ☒

Task 3:

Get file URL ☒

Task 4:

Create Spring Boot upload API ☒

Task 5 (PRO):

Store file URL in DB ☒

CLOUD DAY 4: LOAD BALANCER + SCALING

Goal

By end of today, you will: ☒ Understand load balancing (VERY IMPORTANT)

☒ Handle high traffic systems

☒ Learn auto scaling

☒ Design production-level architecture

✓ PART 1: Problem Without Load Balancer



👉 Suppose:

None

10000 users → 1 server

👉 Result: ✗ Server crashes

✗ Slow response

✗ Downtime

✓ PART 2: What is Load Balancer? 🔥

👉 Load Balancer = Traffic manager

🧠 Simple Idea

None

Users → Load Balancer → Multiple Servers

✓ Example:

None

Load Balancer
/ | \

Server1 Server2 Server3

- ✓ Distributes traffic
 - ✓ Prevents crash
 - ✓ Improves performance
-

✓ PART 3: Types of Load Balancing

✓ 1. Round Robin

None

User1 → Server1

User2 → Server2

User3 → Server3

✓ 2. Least Connections

👉 Send request to least busy server ✓

✓ 3. IP Hash

👉 Same user → same server

✓ PART 4: AWS Load Balancer (ELB) 🔥

👉 AWS provides:

✓ Elastic Load Balancer (ELB)

✓ Types:

Type	Use
Application LB	HTTP/HTTPS ✓
Network LB	High performance
Classic LB	Legacy

👉 You will use: ✓ Application Load Balancer

✓ PART 5: How It Works 🔥

None

User → Load Balancer → EC2 Instances



Spring Boot Apps

✓ Traffic evenly distributed ✓

✓ PART 6: Auto Scaling 🔥 🔥 🔥

👉 Automatically adjusts servers based on traffic

🧠 Example:

None

Low traffic → 1 server

High traffic → 5 servers

✓ Saves cost

✓ Prevents crash

✓ PART 7: Auto Scaling Components

✓ 1. Launch Template

👉 Defines server config

✓ 2. Auto Scaling Group (ASG)

👉 Manages number of EC2 instances

✓ 3. Scaling Policy

👉 Rules like:

None

If CPU > 70% → add server

✅ **PART 8: Step-by-Step Setup** 🔥

✅ **Step 1: Create EC2 AMI**

👉 Create image of your app server

✅ **Step 2: Create Target Group**

👉 Group of servers

✅ **Step 3: Create Load Balancer**

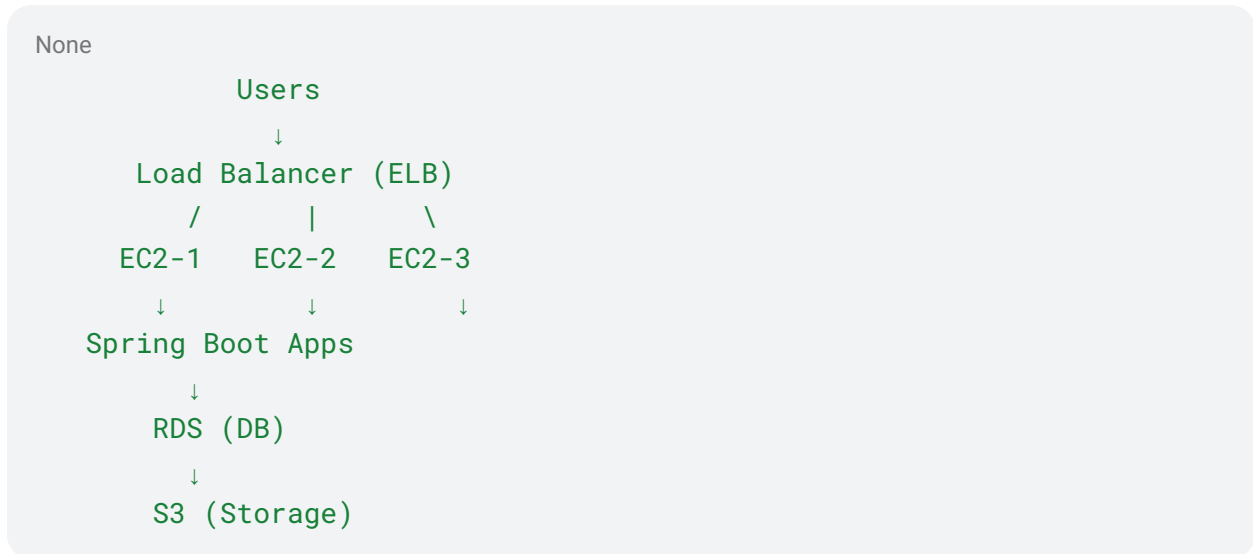
- Select Application LB
 - Attach target group
-

✅ **Step 4: Create Auto Scaling Group**

- Attach load balancer
 - Define min/max instances
-
-

✅ PART 9: Real Architecture 🔥 🔥 🔥

Now your system looks like:



🔥 This is **REAL** production architecture

✅ PART 10: Benefits 🔥

- ✅ High availability
- ✅ Fault tolerance
- ✅ Scalability
- ✅ Better performance

✅ PART 11: What Happens When Server Fails?

👉 Without LB: ❌ App down

👉 With LB: ✅ Traffic moves to healthy servers

✅ System stays alive ✅

✅ PART 12: Health Checks 🔥

👉 Load balancer checks server health

Example:

None
/health API

✅ If server fails → removed from rotation

✅ PART 13: Real Company Example 🔥

Netflix / Amazon

✓ Thousands of servers

✓ Load balancing

✓ Auto scaling

👉 That's why system never crashes ✓

✓ PART 14: Interview Questions 🔥 🔥 🔥

1. What is load balancer?
 2. Types of load balancing?
 3. What is auto scaling?
 4. What is ELB?
 5. What happens if server fails?
 6. What is health check?
-
-

✓ MINI TASK (IMPORTANT)

🔥 Task 1:

Explain: 👉 Why single server fails

🔥 Task 2:

Draw architecture:

None

Client → LB → Servers → DB

Task 3:

Explain:  How auto scaling works

Task 4 (PRO):

Design:  App for 1M users using LB + scaling

CLOUD DAY 5: DOCKER + CI/CD + PRODUCTION PIPELINE

Goal

By end of today, you will: ☒ Understand Docker (VERY IMPORTANT )

☒ Containerize your app ☒

☒ Learn CI/CD pipeline ☒

☒ Understand how real companies deploy apps ☒

☒ PART 1: Problem Without Docker

 Example:

- Works on your laptop ☒
- Fails on server 

 Why?

- Different environments
- Missing dependencies

✓ PART 2: What is Docker? 🐳

👉 Docker = **Containerization tool**

🧠 Simple Understanding

👉 Package your app with:

- Code ✓
 - Dependencies ✓
 - Runtime ✓
-

🧠 Analogy

None

ZIP file → Everything packed

Docker container → Same idea

✓ Runs anywhere

✓ Same behavior everywhere

✓ PART 3: Docker Architecture

None

Docker Image → Blueprint

Docker Container → Running app

-
- ✓ Image = class
 - ✓ Container = object
-

✓ PART 4: Create Dockerfile 🔥

👉 This defines your app

✓ Example (Spring Boot)

Dockerfile

```
FROM openjdk:17
```

```
COPY target/app.jar app.jar
```

```
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Show more lines

✓ This tells Docker:

- Use Java ✓
 - Copy JAR ✓
 - Run app ✓
-

✓ PART 5: Docker Commands 🔥

✓ Build Image

Shell

```
docker build -t myapp .
```

Show more lines

✓ Run Container

Shell

```
docker run -p 8080:8080 myapp
```

Show more lines

✓ Your app runs inside container ✓

✓ PART 6: Why Docker is IMPORTANT?

✓ Same environment everywhere

✓ Easy deployment

✓ Required in most companies

👉 Without Docker: ✗ You are incomplete cloud engineer

✓ PART 7: Docker + Your Architecture 🔥

None

API → Docker Container → Cloud (EC2)

✓ You deploy container, not raw code ✓

✓ PART 8: CI/CD 🔥 🔥 🔥

🧠 What is CI/CD?

👉 Automation pipeline

✓ Flow

None

Code Push → Build → Test → Deploy

✓ No manual deployment ✓

✓ Fast releases ✓

✓ PART 9: Tools

✓ GitHub Actions

✓ Jenkins

✓ GitLab CI

👉 Start with: ✓ GitHub Actions (easy)

PART 10: CI/CD Example (GitHub Actions)

Create `.github/workflows/deploy.yml`

YAML

name: Build and Deploy

on:

push:

branches: ["main"]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v2

- name: Set up Java

uses: actions/setup-java@v3

with:

java-version: '17'

- name: Build

run: mvn clean install

Show more lines

 Automatically builds on push 

✓ PART 11: Full Deployment Flow 🔥 🔥 🔥

```
None
Developer pushes code
    ↓
GitHub Actions runs
    ↓
Builds project
    ↓
Creates Docker image
    ↓
Deploys to server
    ↓
App live ✓
```

👉 This is **real production pipeline**

✓ PART 12: Final Architecture (YOUR LEVEL NOW 🔥 🔥 🔥)

```
None
Frontend (React)
    ↓
Load Balancer
    ↓
Spring Boot (Docker Container)
    ↓
RDS (Database)
    ↓
S3 (Storage)
```

↓
ML Service (optional AI)

🔥 This is **industry-ready architecture**

✓ **PART 13: DevOps Mindset** 🔥

You now understand:

- ✓ Build
 - ✓ Deploy
 - ✓ Scale
 - ✓ Maintain
-

👉 You are no longer just developer

👉 You are:

🔥 **Fullstack + Cloud + DevOps-aware engineer**

✓ **PART 14: Interview Questions** 🔥 🔥 🔥

1. What is Docker?
 2. Difference: Image vs Container?
 3. What is CI/CD?
 4. Why use Docker?
 5. How deployment works in production?
-

MINI TASK (CRITICAL)

Task 1:

Create Dockerfile for your app 

Task 2:

Run Docker container 

Task 3:

Push code to GitHub 

Task 4:

Setup CI/CD pipeline 

Task 5 (PRO):

Deploy Docker app to EC2 

FINAL ACHIEVEMENT

Sameer — BIG milestone:

✓ Backend ✓

✓ AI ✓

✓ System Design ✓

✓ Cloud ✓

✓ Docker ✓

✓ CI/CD ✓